Coarse latent states with learned subpatch moments for multi-resolution ocean emulation

> **Draft status (2026-07-24):** S1 reconstruction and S2 objective selection are complete. The promoted full one-/half-degree S3 result is pending and no quarter-degree result is included.

Abstract

The `SamudraMulti` architecture on `main` maps every physical resolution to a fixed 60×72 processor grid, but its Perceiver encoder compresses each fixed $3^\circ \times 5^\circ$ patch without an explicit resolved-value route and its Perceiver IO decoder must learn spatial routing and channel rendering jointly. Earlier controlled experiments showed that this decoder dominates the unexpected error on the nominally trivial auto-encoding task. A native-grid learned inverse fixes reconstruction but no longer tests the desired scientific regime: a processor state spatially coarser than the observations.

We therefore retain the 60×72 processor grid and replace the representation heads with (i) an area-weighted resolved mean plus 16 learned continuous within-patch moments and (ii) a coordinate-resampling base plus a zero-initialized, position-anchored local attention residual. The inverse is trained jointly, then frozen. A shared processor advances the latent state once per physical time step; a separate boundary encoder supplies exactly one aligned forcing state to each invocation. At matched one-/half-degree reconstruction budget, the learned decoder reduces wet-cell MSE by 62–72% relative to a bilinear-only decoder and retains 13–22 times more target gradient power. In a four-objective dynamics screen, physical loss plus a 0.1-weighted stop-gradient latent-teacher loss wins all aggregate leads and 11 of 12 exact route/lead comparisons. Removing the learned subpatch channels after training increases lead-one raw physical MSE by factors of 2.4–2.9, demonstrating that the processor uses information beyond patch means. The remaining concerns are half-degree lead-one skill and weak velocity high-wavenumber power. The promoted full-scale result will determine whether this coarse representation should replace or remain complementary to the completed native-grid model.

For reproducibility, “`main`” denotes `origin/main` commit `d689f92c`, whose `SamudraMulti` and default model configuration were inspected on 2026-07-24.

Architecture and experimental contract

Property	`SamudraMulti` on `main`	Coarse moment/attention model
Processor grid	Fixed 60×72	Fixed 60×72
Patch encoder	Perceiver compression to one token	Explicit area mean plus learned subpatch moments
Decoder	Perceiver IO learns routing and channels jointly	Coordinate base plus anchored local attention residual
Query role	Position query primarily controls attention	Position anchors logits, conditions values, and enters hidden output directly
Geometry	Added to reconstructive representation	Separate learned processor sidecar
Boundary forcing	Mixed with state before encoding	One separately encoded boundary state per processor step
Autoregression	Decode and re-encode between steps	Encode once and remain latent
Inverse during dynamics	Jointly trainable	Frozen and bitwise audited
Output resolution	Flexible query grid	Flexible coordinate grid

For patch p, physical cell i, channel vector x_{pi}, and spherical area weight a_{pi}, the encoder first computes

$$\mu_p = \frac{\sum_i a_{pi} x_{pi}}{\sum_i a_{pi}}.$$

A coordinate network learns 16 scalar basis functions $\phi_m(r_{pi})$ of normalized within-patch latitude/longitude. Each basis is area-centered and RMS-normalized within the actual input-resolution patch, giving

$$q_{pcm} = \frac{\sum_i a_{pi}(x_{pic} - \mu_{pc})\tilde{\phi}_m(r_{pi})}{\sum_i a_{pi}}.$$

The 160-channel token is

$$z_p = \left[W_\mu \mu_p,\ W_q\ \mathrm{vec}(q_p)\right],$$

with 40 resolved-mean channels and 120 projected moment channels. These are learned coordinate moments, not 16 fixed statistical moments. The same coordinate network operates on 3×5 one-degree patches and 6×10 half-degree patches, so both inputs produce 60×72 tokens.

The decoder is

$$D(z; \ell_o) = B(z; \ell_o) + C(z; \ell_o).$$

B is periodic-longitude physical-coordinate bilinear resampling followed by a shared output-channel projection. C attends over a 3×3 neighborhood of coarse tokens. Its attention logits include a strong distance-to-query anchor; its values receive the unnormalized token and the continuous query-to-token offset; and a direct query residual carries absolute spherical position and source/output scale into the decoded hidden state. Only the key route applies LayerNorm. The value route, base route, and explicit resolved means therefore retain physical amplitude. The correction output is initialized to zero, so optimization begins from B, but the converged correction is not constrained to remain small.

After inverse training, encoder and decoder parameters are frozen and one physical step is

$$z_m = z_{m-1} + \alpha \odot P\left(z_{m-1} + E_b(b_m) + G(\ell_z)\right).$$

P is the shared nonlinear processor, E_b is a separate boundary encoder, and G is a learned, zero-initialized projection of deterministic spherical position and cell-area features. The learned $\alpha \in \mathbb{R}^{1\times160\times1\times1}$ scales processor residual channels; it is unrelated to the decoder correction. A lead N means N autoregressive processor applications and N boundary-encoder calls, advancing the forecast by $N\,dt$. Intermediate states remain latent: they are not decoded and re-encoded.

The selected dynamics objective is

$$L = L_x(D(z_N), x_{t+N}) + 0.1\frac{\sum_{b,c,p} m_{bp}\,(z_{N,bcp} - \mathrm{sg}\,E(x_{t+N})_{bcp})^2}{\sum_{b,c,p} m_{bp}},$$

where m marks wet coarse tokens and sg stops gradients through the target encoding.

Evidence

The S1 inverse was tested with two independent learned-decoder seeds and a matched bilinear-only control. Across all one-/half-degree input/output routes, the learned decoder gives wet-cell MSE 0.097–0.183, versus 0.353–0.482 for bilinear. It retains target gradient-power ratios 0.39–0.70, versus 0.019–0.056, without a patch-seam error spike. The two learned seeds agree closely. Bilinear produces higher synchronized-resolution latent cosine (0.974 versus 0.961) while losing most fine structure, showing why latent agreement alone is not a suitable promotion criterion.

S2 froze the selected seed-15 inverse and trained four matched processor objectives for 768 updates. Full held-out-year validation gives:

Objective (w_x, λ_z)	**Lead 1**	**Lead 2**	**Lead 4**	**Persistence reduction**
Physical only $(1, 0)$	0.1038	0.1322	0.1653	3.3% / 25.5% / 34.6%
Latent only $(0, 1)$	0.1042	0.1313	0.1620	2.9% / 26.1% / 35.8%
Combined $(1, 0.01)$	0.1029	0.1308	0.1636	4.2% / 26.3% / 35.2%
Combined $(1, 0.1)$	**0.1013**	**0.1283**	**0.1608**	**5.7% / 27.7% / 36.3%**

The promoted combined objective is best in 11/12 exact route/lead cells. The exception is one-degree to one-degree at lead four, where latent-only is 1.4% lower. The promoted model beats persistence at every lead-two and lead-four route, but remains 3.7% worse at half-degree same-grid lead one. Aggregate high-wavenumber power ratios are 0.791/0.916/0.336/0.449/0.655 for `thetao`/`so`/`uo`/`vo`/`zos`: scalar fields are credible, but velocity is still too smooth.

Checkpoint audits verify that all 30 inverse tensors remain bit-identical. Zeroing the 120 moment channels while retaining the 40 mean channels increases lead-one raw MSE from 0.136 to 0.391 on the one-degree grid and from 0.221 to 0.519 on the half-degree grid. Same-latent cross-output patch-mean symmetric normalized MSE stays near 0.017 through lead four. Zero boundary forcing increases aggregate lead-four error by 4.6%, and reversed forcing order by 1.0%; the boundary path is causal but not yet strongly used in this short screen.

The principal forecast comparators have deliberately different scopes:

Model	**Processor grid**	**Training scope**	**Lead 1**	**Lead 2**	**Lead 4**
`main` Perceiver model	60×72	full one-degree, historical contract	0.29469†	—	—
Native-grid learned inverse	input grid	full one-/half-degree, 6,392 updates	0.03982	0.05408	0.06595
Coarse moment/attention S2	60×72	768-update objective screen	0.10125	0.12831	0.16083
Coarse moment/attention S3	60×72	full one-/half-degree, 6,392 updates	pending	pending	pending

†The `main` value is its one-degree same-grid lead-one result, not an aggregate over four routes. It is useful historical context but not a controlled single-factor comparison. The native and S3 runs share the one-/half-degree data interval, route schedule, true processor depths, frozen-inverse temporal contract, optimizer-update budget, and validation year; their distinct spatial latent grids are the intended comparison.

S3 promoted full-run result: pending.

Discussion and recommendation

The evidence rejects two simple explanations of the original decoder failure. It is not primarily a shortage of Perceiver decoder latents: increasing them did not resolve the controlled auto-encoding error. It is also not evidence that a coarse latent is intrinsically unusable. Rather, a coarse state needs an explicit low-frequency route, learned phase-sensitive subpatch summaries, and a decoder whose output query is spatially anchored and directly represented in the value/rendering path. The selected decoder is therefore a form of position-anchored direct cross-attention, used as a learned residual around a stable coordinate route.

The S2 result also argues against adding a latent-alignment penalty now. Physical-only training produces the strongest cross-resolution latent agreement at lead four but worse forecasts, while the combined objective improves physical skill and teacher-latent accuracy without collapsing resolution-specific subpatch state. Likewise, the causal moment ablation argues against reverting to a mean-only or purely linear restriction.

The architectural recommendation remains conditional on S3. If the full run materially closes the lead-one and velocity-spectrum gaps while preserving the frozen inverse, promote the patch-moment encoder, continuous anchored hybrid decoder, per-step boundary encoder, and 0.1-weighted latent teacher as the coarse-latent model. If the gaps persist despite the matched update budget, retain the completed native-grid inverse as the production baseline and treat the coarse model as evidence that representation is viable but processor exposure/capacity is insufficient. In either case, do not start quarter-degree validation before reviewing the one-/half-degree endpoint.

Glossary and implementation map

- **`PatchMomentEncoder` / learned moments.** The exact equations above are implemented in `encoder.py`. Configuration: `encoder.patch_moment_count: 16`, `encoder.patch_mean_channels: 40`, and `patch_extent: [3.0, 5.0]` in `model_iterable_inverse_coarse_moment_attention.yaml`.
- **Frozen inverse.** The jointly trained mapping $D \circ E$ from physical state to coarse latent and back. “Frozen” means every `encoder.*` and `decoder.*` tensor is excluded from S2/S3 optimization and checked bitwise against the S1 checkpoint. Configuration: `frozen_model_prefixes: ["encoder.", "decoder."]` in `train_cross_1_halfdeg_coarse_latent_dynamics_full.yaml`.
- **Continuous anchored hybrid decoder.** $D = B + C$, implemented by `ContinuousResampleAttentionResidualDecoder`. Configuration begins with `decoder.continuous_resample_attention_residual: true`; radius 1 means nine candidate coarse tokens, four heads of width 32, and position-bias strength 8.0.
- **Zero-initialized attention residual.** Only C’s final output projection starts at zero. It is free to become large; S1 correction RMS is 0.79–0.81 of prediction RMS. This is distinct from processor residual scale α.
- **Position-anchored direct cross-attention.** The decoder correction adds $-8\|r_q - r_k\|^2$ to attention logits, conditions each value on continuous query-to-token offset, and adds

continuous absolute query/scale features directly to the hidden output. See `ContinuousCoordinateAttentionCorrection`.

- **LayerNorm concern.** `content_norm` normalizes only attention keys. Values consume the unnormalized latent token; the bilinear base and resolved-mean route are also unnormalized. This keeps a direct amplitude-bearing path while allowing normalized similarity matching.
- **Geometry sidecar.** `encoder.geometry_mode: sidecar` means absolute position and grid scale are not added to reconstructive encoder content. Instead, `ProcessorGeometryConditioner` supplies them to each processor call.
- **Separate boundary encoder.** `boundary_encoder` maps exactly the forcing state b_m aligned with transition m onto the latent grid. It is called once per processor application; see `SamudraMulti.process` and `SamudraMulti.latent_rollout`.
- **Latent autoregression.** Encode once, apply the processor N times, and decode selected latent states. A lead N is not one processor call with N boundary states. Code: `SamudraMulti.latent_rollout`.
- **Processor residual scale α.** A learned tensor of shape `[1,160,1,1]` used in $z + \alpha \odot P(\cdot)$. It is initialized to zero by `processor_residual: true`; S2 $(1, 0.1)$ has mean absolute value 0.0344.
- **Physical and latent objectives.** `physical_forecast_loss_weight` is w_x; `latent_teacher_loss_weight` is λ_z. The target latent is a no-gradient encoding of x_{t+N}, masked at wet coarse tokens. See `SamudraMulti.forward` and `SamudraMulti.latent_teacher_loss`.
- **Mean-only-initial ablation.** An inference-time causal intervention, not a separately optimized OM4 model: zero the 120 moment channels of the initial latent, retain the 40 mean channels, then run the same trained processor and boundary sequence. Implementation: `audit_coarse_dynamics.py`.
- **Flexible output resolution.** The decoder accepts requested latitude and longitude arrays and evaluates $B + C$ at those coordinates. Flexibility does not imply that arbitrary fine detail can be recovered from a coarse input.
- **S1/S2/S3.** S1 trains and audits the inverse; S2 freezes it and selects the dynamics objective in a short matched screen; S3 trains a fresh processor/boundary path at the full one-/half-degree update budget. The authoritative ledger is `coarse_latent_highres_dynamics_results.md`.